

Boutique en ligne Stubborn

Documentation de l'application e-commerce Symfony

Don't compromise on your look

Devoir CEF - Réaliser un site e-commerce avec Symfony
Symfony 7.4 LTS · MySQL 8 · Stripe (mode test)
Octobre 2026

Sommaire

1. Présentation du projet
2. Parcours utilisateur et pages
3. Architecture technique
4. Modèle de données
5. Sécurité
6. Algorithmes du panier et du paiement
7. Intégration de Stripe
8. Tests unitaires
9. Installation, lancement et achat test
10. Gestion des versions
11. Annexe : référence du code

1. Présentation du projet

Le client

Société	Stubborn - marque de sweat-shirts
Adresse	Piccadilly Circus, London W1J 0DA, Royaume-Uni
Contact	stubborn@blabla.com
Slogan	Don't compromise on your look

Objectif

Stubborn souhaite une boutique en ligne permettant à ses clients de consulter sa collection de sweat-shirts, de choisir une taille, de constituer un panier et de régler leur commande en ligne. Un back-office permet à l'administrateur de gérer le catalogue.

Périmètre réalisé

- Application Symfony 7.4 s'appuyant sur une base MySQL 8.
- Authentification avec le composant Security, rôles client (`ROLE_USER`) et administrateur (`ROLE_ADMIN`).
- Inscription avec e-mail d'activation ; le lien d'activation connecte l'utilisateur et le renvoie sur l'accueil.
- Réinitialisation du mot de passe oublié (lien présent sur le wireframe de connexion).
- Catalogue de 10 sweat-shirts, filtre par fourchette de prix, fiche produit avec choix de la taille.
- Panier en session : total, retrait d'un article, contrôle du stock par taille.
- Classe service `StripeService` : paiement Stripe Checkout en mode développement (bac à sable).
- Back-office `/admin` : ajout, modification (prix, image, mise en avant, stock par taille) et suppression.
- 33 tests unitaires (panier, filtre de prix, service Stripe, achat complet) exécutés au lancement de l'application.

Les sweat-shirts

Chaque sweat-shirt est disponible en tailles XS, S, M, L et XL avec au moins 2 exemplaires par taille ; le prix est identique pour toutes les tailles. Les modèles marqués ★ sont mis en avant sur l'accueil.

Nom	Prix	Nom	Prix
Blackbelt ★	29,90 €	Snow	32,00 €
BlueBelt	29,90 €	Greyback	28,50 €
Street	34,50 €	BlueCloud	45,00 €
Pokeball ★	45,00 €	BornInUsa ★	59,90 €
PinkLady	29,90 €	GreenSchool	42,20 €

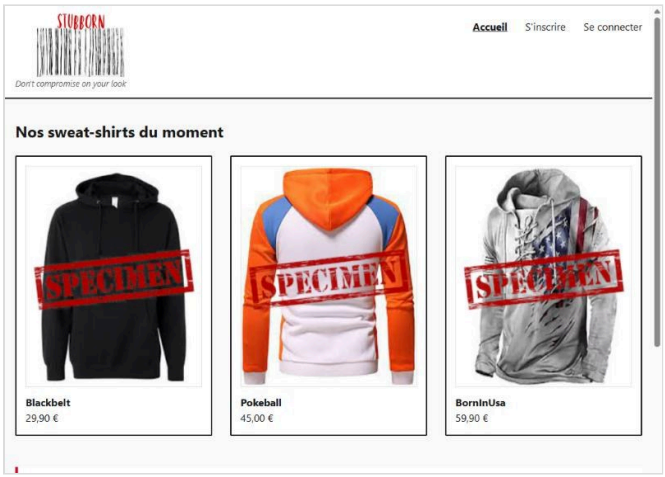
2. Parcours utilisateur et pages

Les vues respectent les wireframes fournis (dossier `docs/wireframes` du dépôt). Dans le menu, le lien de la page courante est affiché **en gras et souligné**.

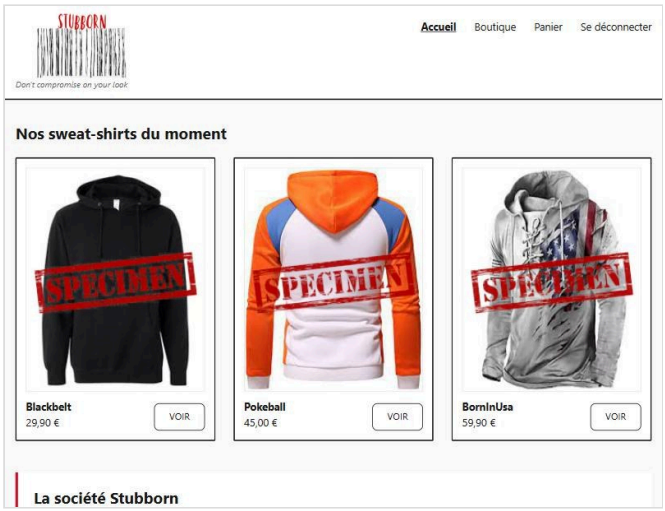
Route	Page	Accès
`/`	Accueil : 3 sweat-shirts mis en avant, présentation de la société	Tous
`/register`	Inscription + e-mail d'activation	Visiteur
`/login`	Connexion, liens « Mot de passe oublié » et « Créer un compte »	Visiteur
`/products`	Boutique, filtre par prix	Connecté
`/product/{id}`	Fiche produit, choix de la taille, ajout au panier	Connecté
`/cart`	Panier, total, retrait, « Finaliser ma commande »	Connecté
`/admin`	Back-office	Administrateur

Accueil et menu dynamique

Un visiteur voit le menu *Accueil / S'inscrire / Se connecter* et les sweat-shirts mis en avant sans bouton « Voir ». Une fois connecté, le menu devient *Accueil / Boutique / Panier / Se déconnecter* et les boutons « Voir » apparaissent.



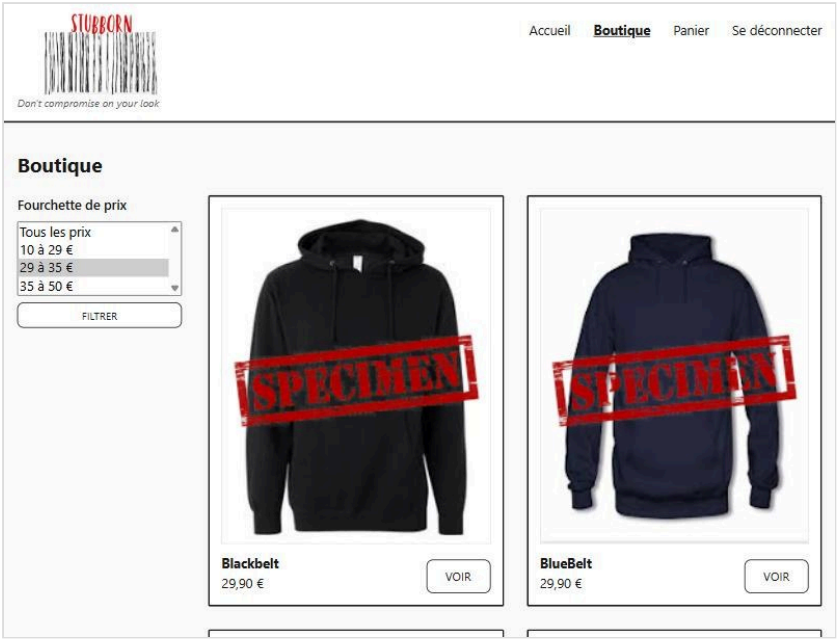
Accueil - visiteur



Accueil - utilisateur connecté

Boutique et filtre par prix

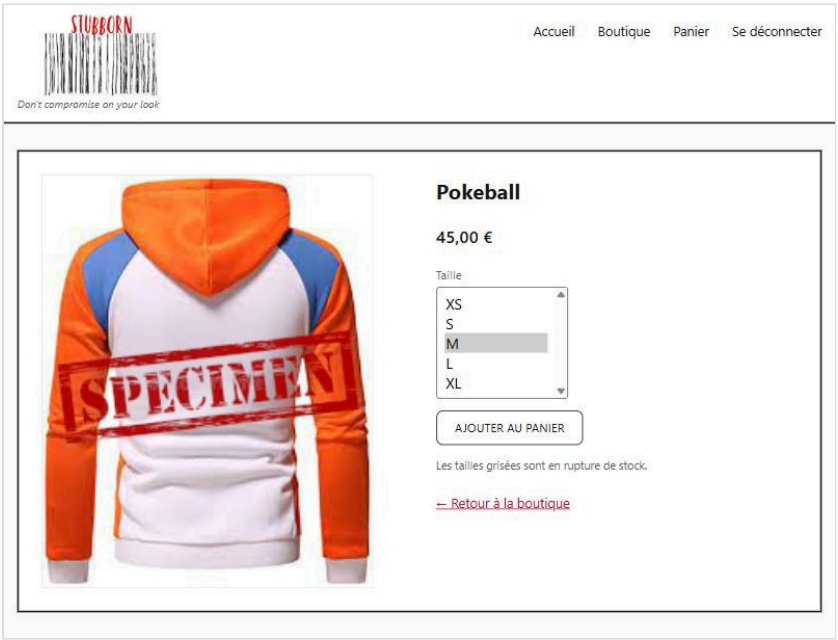
La page `/products` liste tous les sweat-shirts (image, nom, prix, bouton « Voir »). Le filtre propose les fourchettes du cahier des charges ; il s'applique dès la sélection. Les bornes basses sont incluses et les bornes hautes exclues (sauf 50 €) pour qu'un prix n'appartienne jamais à deux fourchettes.



Boutique filtrée sur la fourchette 29 à 35 €

Fiche produit

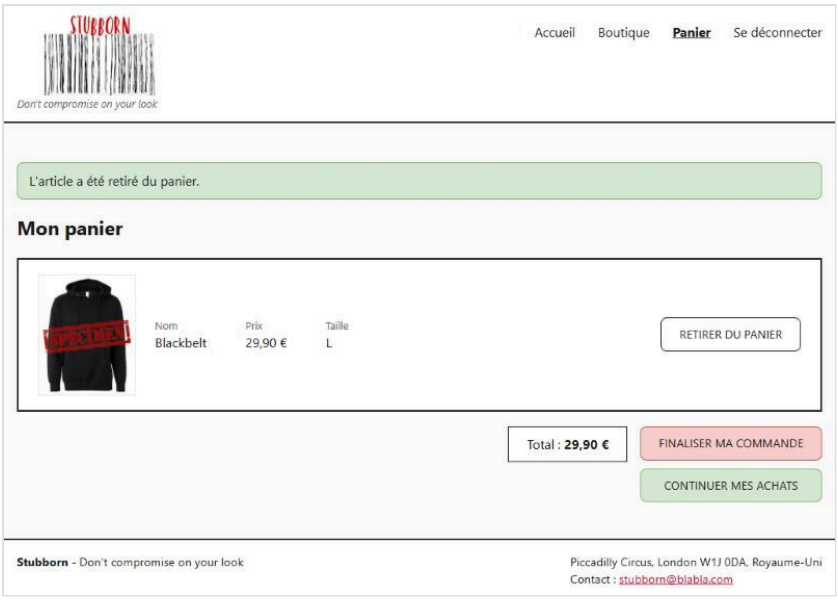
La fiche affiche l'image, le nom et le prix. La liste des tailles présélectionne M (comme le wireframe) ; les tailles en rupture de stock sont désactivées.



Fiche produit

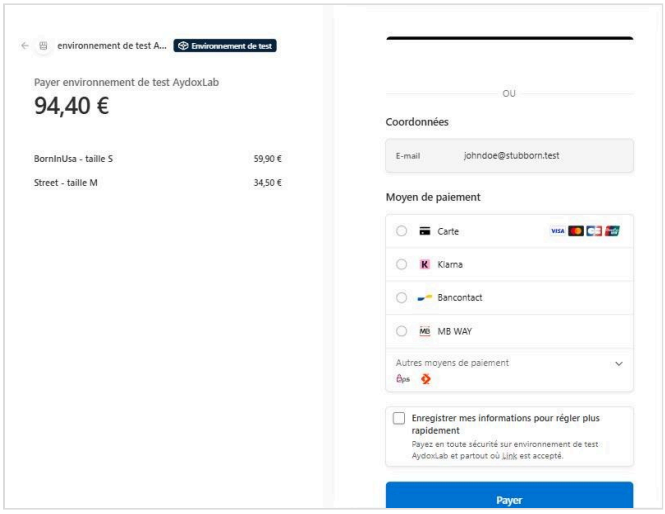
Panier

Le panier liste les articles (image, nom, prix, taille), calcule le total et permet de retirer un article ; le total s'adapte immédiatement. « Continuer mes achats » ramène à la boutique, « Finaliser ma commande » lance le paiement Stripe.

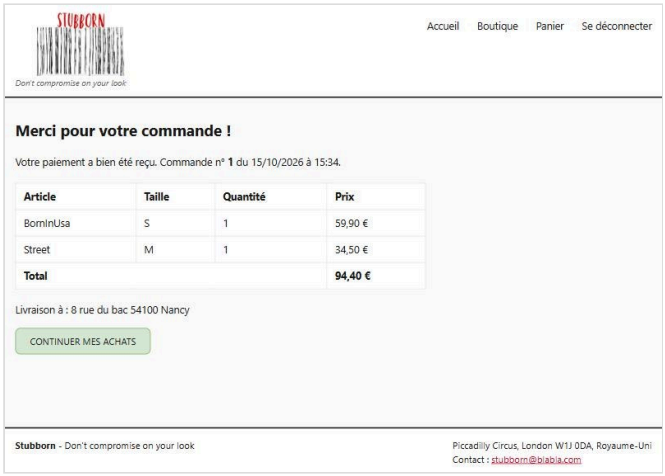


Panier

Paiement et confirmation



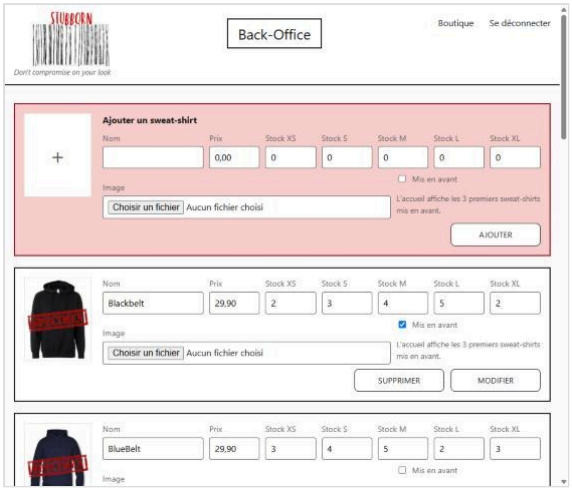
Page de paiement Stripe (environnement de test)



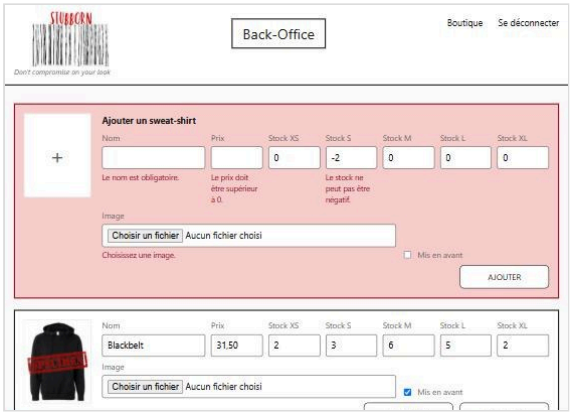
Confirmation de la commande

Back-office

Réservé aux administrateurs, il reprend la structure du wireframe : une ligne d'ajout (en rouge) puis une ligne par sweat-shirt. Chaque ligne est un formulaire permettant de modifier le nom, le prix, le stock de chaque taille, l'image et la mise en avant, ou de supprimer le sweat-shirt. Les erreurs de saisie sont affichées sous chaque champ.



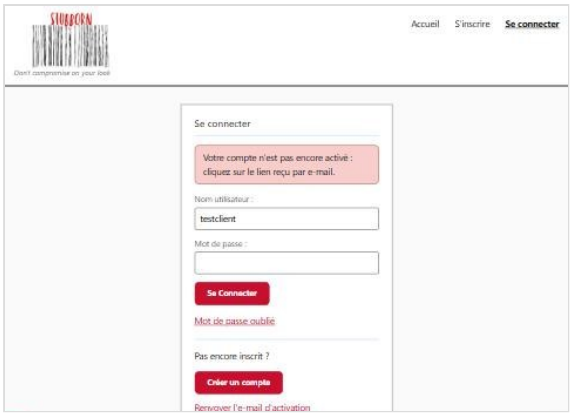
Back-office



Validation des données saisies

Inscription et activation du compte

Le formulaire d'inscription demande le nom d'utilisateur, l'adresse e-mail, l'adresse de livraison et le mot de passe (deux fois). Le compte est créé inactif et un e-mail contenant un lien signé, valable une heure, est envoyé. Tant que ce lien n'a pas été suivi, la connexion est refusée avec un message explicite et la possibilité de renvoyer l'e-mail.



Connexion refusée pour un compte non activé

Affichage mobile



Page de connexion sur smartphone

3. Architecture technique

Technologies

Couche	Choix	Rôle
Framework	Symfony 7.4 LTS (PHP 8.2)	Routage, contrôleurs, injection de dépendances, formulaires, sécurité
Base de données	MySQL 8 + Doctrine ORM 3	Persistance, migrations
Vues	Twig + CSS	Gabarits, thème de formulaire, mise en page responsive
Paiement	stripe/stripe-php (SDK officiel)	Stripe Checkout en mode test
E-mails	Symfony Mailer + Mailpit	Activation du compte, mot de passe oublié
Comptes	SymfonyCasts VerifyEmail, ResetPassword	Liens signés et jetons de réinitialisation
Tests	PHPUnit 11	Tests unitaires

Concernant Stripe, il n'existe pas de bundle Symfony officiel maintenu par Stripe ; le choix s'est porté sur le SDK PHP officiel, encapsulé dans une classe service de l'application.

Organisation du code (MVC)

```
src/
├── Controller/   Reçoivent la requête, appellent les services, rendent une vue
├── Entity/       Product, User, Order, OrderItem, ResetPasswordRequest
├── Enum/         Size (XS, S, M, L, XL)
├── Form/         Formulaires et règles de validation
├── Model/        CartItem (ligne de panier), PriceRange (fourchette de prix)
├── Repository/   Requêtes Doctrine
├── Security/     EmailVerifier, UserChecker
├── Service/      CartService, StripeService, CheckoutService, ProductImageUploader
└── Twig/         Filtres d'affichage des prix
templates/        Vues Twig (une par page + base.html.twig)
tests/            Tests unitaires
migrations/       Migration du schéma MySQL
```

Services principaux

Service	Responsabilité
`CartService`	Panier en session : ajout, regroupement, retrait, total en centimes, contrôle du stock.
`StripeService`	Création de la session Stripe Checkout à partir d'une commande, vérification du paiement.
`CheckoutService`	Orchestration d'un achat : panier → commande → paiement → confirmation (stock mis à jour, articles achetés retirés du panier).
`ProductImageUploader`	Enregistrement sécurisé des images envoyées depuis le back-office.
`EmailVerifier`	Envoi et vérification du lien d'activation.

Les services sont injectés automatiquement (autowiring). Les contrôleurs restent fins : la logique métier est dans les services, ce qui permet de la tester sans navigateur ni base de données.

4. Modèle de données

Le schéma est créé par la migration `migrations/Version20261015131758.php`.

user

Colonne	Type	Description
id	INT	Clé primaire
name	VARCHAR(50), unique	Nom d'utilisateur, identifiant de connexion
email	VARCHAR(180), unique	Adresse de messagerie
password	VARCHAR(255)	Mot de passe haché
delivery_address	VARCHAR(255), nullable	Adresse de livraison (client uniquement)
roles	JSON	`ROLE_USER`, `ROLE_ADMIN`
is_verified	BOOLEAN	Compte activé par e-mail

product

Colonne	Type	Description
id	INT	Clé primaire
name	VARCHAR(100)	Nom du sweat-shirt
price	DECIMAL(8,2)	Prix en euros, identique pour toutes les tailles
image	VARCHAR(255)	Fichier dans `public/images/products`
featured	BOOLEAN	Mis en avant sur l'accueil
stock_xs … stock_xl	INT × 5	Stock pour chaque taille

order et order_item

Table	Colonnes principales
order	customer_id → user, status (`pending`, `paid`, `canceled`), total_in_cents, delivery_address, stripe_session_id (unique), created_at, paid_at
order_item	order_id → order, product_id → product (SET NULL à la suppression), product_name, size, quantity, unit_price_in_cents

Le nom et le prix sont recopiés dans chaque ligne de commande : l'historique reste exact même si l'administrateur modifie ou supprime ensuite le sweat-shirt. Les montants de commande sont stockés en centimes (entiers) pour éviter les erreurs d'arrondi.

La table `reset_password_request` stocke les jetons (hachés) de réinitialisation de mot de passe.

5. Sécurité

Authentification et permissions

- Composant **Security** : fournisseur d'utilisateurs Doctrine (connexion par nom d'utilisateur), authentification par formulaire (`form_login`) avec jeton CSRF, redirection systématique vers l'accueil après connexion.
- Mots de passe hachés avec l'algorithme `auto` (bcrypt/argon2) et ré-hachage automatique si l'algorithme évolue.
- `UserChecker` : refuse la connexion des comptes non activés.
- Règles `access_control` : `/admin` exige `ROLE_ADMIN` ; `/products` , `/product` , `/cart` et `/checkout` exigent `ROLE_USER` . Le back-office est en plus protégé par l'attribut `#[IsGranted('ROLE_ADMIN')]` .
- Un utilisateur connecté qui tente d'ouvrir une page non autorisée (un client sur `/admin`) est renvoyé vers l'accueil avec un message, au lieu d'une page d'erreur.

Style défensif

- **CSRF** sur tous les formulaires et actions sensibles (connexion, déconnexion, retrait du panier, paiement, suppression).
- **Limitation des tentatives** : 5 essais de connexion par minute (protection contre la force brute), 3 renvois de l'e-mail d'activation par quart d'heure et par adresse IP.
- **Clé Stripe de test obligatoire** : le service refuse toute clé réelle (`sk_live_`), aucun paiement réel ne peut être encaissé.
- **Validation** côté serveur de toutes les saisies (contraintes Symfony Validator) ; mot de passe d'au moins 8 caractères avec lettres et chiffres, refusé s'il figure dans une fuite connue.
- Lien d'activation **signé (HMAC) et expirant** ; la signature est vérifiée à chaque utilisation, un lien modifié est refusé.
- « Mot de passe oublié » : le même message est affiché que le compte existe ou non (pas d'énumération des comptes) ; le jeton est retiré de l'URL.
- **Paiement revérifié auprès de l'API Stripe** : l'URL de retour ne suffit pas à valider une commande. La confirmation vérifie aussi que la commande appartient au client connecté et n'est traitée qu'une fois.
- Stock revérifié au moment de créer la commande ; quantités et tailles contrôlées.
- Images envoyées : type MIME (JPEG, PNG, WebP) et taille (2 Mo) contrôlés, nom de fichier régénéré.
- Échappement automatique des vues Twig (protection XSS) et requêtes Doctrine paramétrées (protection injection SQL).
- Secrets (base de données, clés Stripe) dans `.env.local` , exclu du dépôt Git.

6. Algorithmes du panier et du paiement

Ajouter un sweat-shirt au panier

```
FONCTION ajouter(produit, taille, quantité = 1)
    SI produit n'est pas enregistré OU quantité < 1 ALORS erreur
    panier ← lire la session["cart"]
    clé ← produit.id + "-" + taille            // une ligne par couple produit/taille
    nouvelleQuantité ← panier[clé].quantité (0 si absente) + quantité
    SI nouvelleQuantité > produit.stock(taille) ALORS erreur "stock insuffisant"
    SI nouvelleQuantité > 10 ALORS erreur "quantité maximale atteinte"
    panier[clé] ← { produit.id, taille, nouvelleQuantité }
    écrire panier dans la session["cart"]
FIN
```

Calculer le total

```
FONCTION total()
    lignes ← lignes du panier, produits relus en base (prix à jour)
    ignorer les lignes dont le produit a été supprimé
    total ← 0
    POUR CHAQUE ligne DANS lignes
        total ← total + prixEnCentimes(ligne.produit) × ligne.quantité
    RETOURNER total                              // en centimes, sans erreur d'arrondi
FIN
```

Acheter

```
1. « Finaliser ma commande » (POST + jeton CSRF)
   SI panier vide ALORS erreur
   POUR CHAQUE ligne : SI quantité > stock actuel ALORS erreur
   annuler les commandes "pending" précédentes du client (paiements abandonnés)
   créer la commande (statut "pending") avec une ligne par article, enregistrer
2. StripeService.créerSession(commande)  →  URL de paiement Stripe
   SI Stripe renvoie une erreur ALORS commande "canceled", retour au panier
   mémoriser l'identifiant de session dans la commande, rediriger le client
3. Retour de Stripe sur /checkout/success?session_id=…
   commande ← chercher par session_id
   SI commande introuvable OU client différent ALORS erreur
   SI commande déjà payée ALORS afficher la confirmation (pas de double traitement)
   SI Stripe ne confirme pas payment_status = "paid" ALORS erreur
   POUR CHAQUE ligne : stock(taille) ← max(0, stock - quantité)
   commande.statut ← "paid"
   retirer du panier les articles achetés (un article ajouté entre-temps reste)
   afficher la confirmation
4. Retour sur /checkout/cancel : commande "canceled", panier conservé
```

7. Intégration de Stripe

L'application utilise **Stripe Checkout** : la page de paiement est hébergée par Stripe, aucune donnée bancaire ne transite par le serveur de la boutique.

La classe service StripeService

Méthode	Rôle
`isConfigured()` / `isTestMode()`	Vérifie qu'une clé secrète est renseignée et qu'il s'agit d'une clé de test (`sk_test_`) ; toute autre clé est refusée avant le moindre appel à Stripe.
`createCheckoutSession(Order, successUrl, cancelUrl)`	Crée la session : une ligne par article (nom + taille, prix unitaire en centimes, quantité, devise EUR), e-mail du client, URL de retour.
`retrieveCheckoutSession(id)`	Relit la session auprès de l'API.
`isCheckoutSessionPaid(id)`	Retourne vrai si Stripe indique `payment_status = paid` .

Mode développement (bac à sable)

Les clés de test se trouvent dans le tableau de bord Stripe (*Développeurs → Clés API*, mode test) et la clé secrète se renseigne dans `.env.local` (`STRIPE_SECRET_KEY`) ; la clé publiable n'est pas nécessaire car la redirection vers Stripe Checkout se fait côté serveur. Les paiements sont fictifs ; ils apparaissent dans le tableau de bord en mode test.

Limite connue : la commande est validée au retour du client sur la boutique. Si le client fermait son navigateur juste après avoir payé, la commande resterait « en attente » ; en production, on ajouterait un webhook Stripe (`checkout.session.completed`) pour la valider côté serveur. Ce n'est pas nécessaire pour la démonstration en local.

Carte de test	Résultat
`4242 4242 4242 4242`	Paiement accepté
`4000 0000 0000 9995`	Paiement refusé (fonds insuffisants)
`4000 0025 0000 3155`	Authentification 3D Secure demandée

Date d'expiration : n'importe quelle date future (ex. 12/30) ; CVC : trois chiffres quelconques.

> Achat test réalisé lors de la recette : panier BornInUsa (S) + Street (M) = 94,40 €, paiement par la carte 4242, commande n° 1 enregistrée au statut « paid », stock décrémenté d'un exemplaire pour chaque article, panier vidé. Un paiement abandonné a également été vérifié : commande au statut « canceled », panier conservé. Un nouvel achat (Snow, taille S, 32,00 €) a été refait avec succès après la relecture finale du code.

8. Tests unitaires

Les tests se trouvent dans `tests/Service`. Ils n'ont besoin ni de base de données ni de connexion Internet :

- le dépôt de produits et l'EntityManager sont simulés ; la session est une session en mémoire ;
- pour Stripe, le vrai SDK est utilisé mais son client HTTP est remplacé par un faux client (`tests/Support/FakeStripeHttpClient.php`) qui enregistre les requêtes envoyées et renvoie des réponses préparées. On vérifie ainsi précisément ce qui serait envoyé à Stripe.

Exécution au lancement

La commande `composer start` exécute les tests puis démarre le serveur ; les résultats s'affichent dans la console. Les tests seuls se lancent avec `composer test`.

```
Achat test
 ✓ Achat complet : panier, commande, paiement Stripe, confirmation
 ✓ Recharger la confirmation ne décompte pas le stock deux fois
 ✓ Un paiement non abouti ne valide pas la commande
 ✓ Un autre client ne peut pas valider la commande
 ✓ Un panier vide ne peut pas être commandé
 ✓ Le stock est revérifié avant le paiement
 ✓ Un article ajouté pendant le paiement reste dans le panier
 ✓ Une nouvelle commande annule la commande abandonnée précédente
 ✓ Une erreur Stripe annule la commande en attente

Filtre par prix
 ✓ Les trois fourchettes du cahier des charges sont proposées
 ✓ Chaque sweat-shirt de la collection tombe dans une seule fourchette
 ✓ Les bornes sont gérées sans chevauchement (29 € et 35 €)
 ✓ Une valeur inconnue dans l'URL désactive le filtre

Panier
 ✓ Un nouveau panier est vide
 ✓ Ajout d'un sweat-shirt au panier
 ✓ Le même sweat-shirt dans la même taille est regroupé sur une ligne
 ✓ Le même sweat-shirt dans deux tailles donne deux lignes
 ✓ Le total est calculé sur toutes les lignes
 ✓ Retirer un article met à jour le total
 ✓ Le retrait ne concerne que la taille choisie
 ✓ Impossible d'ajouter plus que le stock disponible
 ✓ Impossible d'ajouter une taille en rupture
 ✓ La quantité doit être positive
 ✓ Le panier est conservé en session
 ✓ Un sweat-shirt supprimé est ignoré
 ✓ Vider le panier

Service Stripe
 ✓ Détection de la clé et du mode test
 ✓ La session de paiement décrit la commande (montants en centimes, EUR)
 ✓ Une session payée est reconnue
 ✓ Une session non payée est refusée
 ✓ Une commande vide ne peut pas être payée
 ✓ Une clé manquante donne une erreur explicite
 ✓ Une clé Stripe réelle (sk_live_) est refusée

OK (33 tests, 96 assertions)
```

9. Installation, lancement et achat test

Prérequis

- PHP 8.2 ou plus (extensions `pdo_mysql`, `curl`, `mbstring`, `ctype`, `iconv`) - XAMPP convient ; activer `extension=zip` dans le `php.ini` pour `composer install`
- Composer, MySQL 8, un compte Stripe en mode test, Mailpit (serveur SMTP de test)

Installation

```
git clone https://github.com/OrionLazu/symfony-CEF.git
cd symfony-CEF
composer install
```

Créer le fichier `.env.local` :

```
DATABASE_URL="mysql://UTILISATEUR:MOT_DE_PASSE@127.0.0.1:3306/stubborn?
serverVersion=8.0.32&charset=utf8mb4"
STRIPE_SECRET_KEY=sk_test_...
```

Créer la base, le schéma et charger les données de démonstration :

```
composer db:init
```

Lancement

```
mailpit            # e-mails : http://localhost:8025
composer start     # tests puis serveur : http://127.0.0.1:8000
```

Comptes de démonstration

Rôle	Nom d'utilisateur	Mot de passe
Administrateur	`admin`	`StubbornAdmin2026`
Client	`johndoe`	`StubbornClient2026`

Effectuer un achat test

1. Se connecter avec `johndoe`, aller dans la Boutique, cliquer sur « Voir ».
2. Choisir une taille, « Ajouter au panier ».
3. Dans le panier, « Finaliser ma commande ».
4. Sur la page Stripe (environnement de test), payer avec `4242 4242 4242 4242`, `12/30`, `123`.
5. La page de confirmation affiche la commande ; le stock est mis à jour et les articles achetés sont retirés du panier.

10. Gestion des versions

Le code est versionné avec Git et publié sur GitHub. Chaque fonctionnalité est développée sur une branche `feature/…` (ou `fix/…` pour les corrections de recette), puis fusionnée dans `develop` ; `main` contient la version livrée.

```
* Documenter le gestionnaire d'accès refusé et préciser l'achat test dans le README
* Mettre à jour le README et la configuration après la vérification complète
* Corriger les remarques de relecture : message du prix, code inutilisé, aide du back-office
* Fiabiliser le paiement : clé de test obligatoire, commandes abandonnées annulées, panier
préservé
* Renforcer la sécurité : limitation des tentatives, déconnexion protégée, accès refusé redirigé
*   Fusionner feature/documentation dans develop
|\
| * Rédiger le README : installation, lancement, comptes et achat test
| * Ajouter la documentation PDF de l'application et les captures d'écran
| * Compléter les commentaires de documentation du code
|/
*   Fusionner fix/corrections-recette dans develop
|\
| * Présélectionner la taille M sur la fiche produit
| * Afficher une erreur de validation quand un champ texte obligatoire est vide
| * Valider le lien d'activation à partir de la requête
| * Installer HttpClient requis par la vérification des mots de passe compromis
|/
*   Fusionner feature/migration-mysql dans develop
|\
| * Ajouter la migration du schéma MySQL
|/
*   Fusionner feature/tests-unitaires dans develop
|\
| * Tester le panier, le service Stripe et un achat complet
|/
*   Fusionner feature/back-office dans develop
|\
| * Créer le back-office de gestion des sweat-shirts
|/
*   Fusionner feature/paiement-stripe dans develop
|\
| * Intégrer le paiement Stripe Checkout en mode test
|/
*   Fusionner feature/boutique-et-panier dans develop
|\
| * Gérer le panier en session
| * Afficher l'accueil, la boutique filtrable et la fiche produit
| * Créer la mise en page commune et le menu dynamique
|/
*   Fusionner feature/authentification dans develop
|\
| * Ajouter la réinitialisation du mot de passe oublié
| * Ajouter l'inscription avec e-mail d'activation du compte
| * Mettre en place la connexion avec le composant Security
|/
*   Fusionner feature/modele-de-donnees dans develop
|\
```

11. Annexe : référence du code

Cette annexe est générée automatiquement à partir des commentaires de documentation (docblocks) des classes de `src/` : description de chaque classe et de ses méthodes publiques, avec la route associée pour les contrôleurs. Les accesseurs simples (get/set) sont omis.

Controller

AdminController `/admin`

Back-office : ajout, modification et suppression des sweat-shirts.

- `index(Request $request, ProductRepository $productRepository): Response` - Une seule page affiche le formulaire d'ajout et un formulaire par sweat-shirt. Chaque formulaire porte un nom distinct (product_new, product_12…) : seul celui envoyé est traité.
- `delete(Product $product, Request $request): Response` `/product/{id<\d+>}/delete` - Supprime un sweat-shirt (formulaire POST protégé par un jeton CSRF) et son image.

CartController

Page panier : liste des articles, total, retrait d'un article.

- `index(CartService $cart): Response` `/cart` - Affiche le contenu du panier et son montant total.
- `remove(int $id, string $size, Request $request, CartService $cart): Response` `/cart/remove/{id<\d+>}/{size}` - Retire une ligne du panier (un sweat-shirt dans une taille) ; le total est recalculé à l'affichage.

CheckoutController

Règlement de la commande avec Stripe Checkout.

- `checkout(Request $request): Response` `/checkout` - Bouton « Finaliser ma commande » : crée la commande puis redirige vers la page de paiement Stripe.
- `success(Request $request): Response` `/checkout/success` - Retour de Stripe après paiement : vérification auprès de Stripe puis confirmation de la commande.
- `cancel(Request $request): Response` `/checkout/cancel` - Retour de Stripe quand le client abandonne le paiement : le panier est conservé.

HomeController

Page d'accueil de la boutique.

- `index(ProductRepository $productRepository): Response` `/` - Page d'accueil : présentation de Stubborn et sweat-shirts mis en avant. Les boutons « Voir » ne sont affichés qu'aux utilisateurs connectés (voir le template).

ProductController

Boutique : liste des sweat-shirts et fiche produit (réservées aux utilisateurs connectés).

- `index(Request $request, ProductRepository $productRepository): Response` `/products` - Liste de tous les sweat-shirts, filtrable par fourchette de prix (?range=10-29).
- `show(Product $product, Request $request, CartService $cart): Response` `/product/{id<\d+>}` - Fiche d'un sweat-shirt avec choix de la taille et ajout au panier.

RegistrationController

Inscription des clients et activation de leur compte par e-mail.

- `register(Request $request, UserPasswordHasherInterface $passwordHasher, EntityManagerInterface $entityManager, LoggerInterface $logger): Response` `/register` - Inscription d'un client : le compte est créé inactif et un lien d'activation est envoyé par e-mail.
- `verifyUserEmail(Request $request, UserRepository $userRepository, Security $security): Response` `/verify/email` - Lien reçu par e-mail : active le compte, connecte l'utilisateur et le renvoie sur l'accueil.
- `resend(Request $request, UserRepository $userRepository, LoggerInterface $logger, RateLimiterFactory $verifyResendLimiter): Response` `/verify/resend` - Renvoie un lien d'activation (depuis la page de connexion).

ResetPasswordController `/reset-password`

Parcours « Mot de passe oublié » présent sur le wireframe de connexion.

- `request(Request $request, MailerInterface $mailer, UserRepository $userRepository, LoggerInterface $logger, string $senderEmail): Response` - Étape 1 : saisie de l'adresse e-mail.
- `checkEmail(): Response` `/check-email` - Étape 2 : confirmation d'envoi (toujours affichée, que le compte existe ou non).
- `reset(Request $request, UserPasswordHasherInterface $passwordHasher, ?string $token): Response` `/reset/{token}` - Étape 3 : choix du nouveau mot de passe depuis le lien reçu.

SecurityController

Connexion et déconnexion (formulaire géré par le composant Security).

- `login(AuthenticationUtils $authenticationUtils): Response` `/login` - Formulaire de connexion. Le traitement de l'envoi est assuré par le composant Security (form_login).
- `logout(): never` `/logout` - Cette méthode n'est jamais exécutée : la déconnexion est interceptée par le pare-feu.

DataFixtures

AppFixtures

Données de démonstration : la collection fournie avec le brief et deux comptes de test.

Entity

Order

Commande passée par un client.

- `markAsPaid(): static` - Passe la commande au statut « payée » et horodate le paiement.
- `markAsCanceled(): static` - Passe la commande au statut « annulée » (paiement abandonné).
- `addItem(OrderItem $item): static` - Ajoute une ligne et recalcule le total de la commande.

OrderItem

Ligne de commande : un sweat-shirt dans une taille donnée.

Product

Sweat-shirt vendu sur la boutique.

- `setPrice(string|int|float|null $price): static` - Accepte un nombre ou une chaîne ("29,90" ou "29.90") et la normalise au format DECIMAL. Une valeur vide ou invalide est conservée vide pour que la validation la signale.
- `getPriceInCents(): int` - Prix en centimes : toutes les opérations de calcul (panier, Stripe) travaillent en entiers.

ResetPasswordRequest

Demande de réinitialisation de mot de passe (lien « Mot de passe oublié »).

User

Utilisateur de la boutique.

- `getUserIdentifier(): string` - Identifiant utilisé par le composant Security : le nom d'utilisateur.

Enum

Size

Tailles proposées pour les sweat-shirts Stubborn.

Form

AddToCartType

Choix de la taille sur la fiche d'un sweat-shirt. Les tailles en rupture de stock sont affichées mais désactivées.

ChangePasswordFormType

Saisie du nouveau mot de passe après clic sur le lien de réinitialisation.

ProductType

Formulaire du back-office : une ligne par sweat-shirt (ajout ou modification).

RegistrationFormType

Formulaire d'inscription d'un client (champs du wireframe « S'inscrire »).

ResetPasswordRequestFormType

Demande de lien de réinitialisation (« Mot de passe oublié »).

Model

CartItem

Ligne du panier telle qu'affichée à l'utilisateur.

- `key(int $productId, Size $size): string` - Clé unique d'une ligne : un même sweat-shirt dans deux tailles fait deux lignes.

PriceRange

Fourchette de prix proposée dans le filtre de la boutique.

- `all(): array` - Fourchettes définies par le cahier des charges : 10 à 29 €, 29 à 35 €, 35 à 50 €. Les bornes basses sont incluses et les bornes hautes exclues (sauf 50 €), pour qu'un prix ne tombe jamais dans deux fourchettes à la fois.
- `fromKey(?string $key): ?self` - Retourne la fourchette correspondant à la clé reçue dans l'URL, ou null si elle est inconnue.
- `contains(float $price): bool` - Indique si un prix appartient à cette fourchette.

Repository

OrderItemRepository

Accès aux lignes de commande en base de données.

OrderRepository

Accès aux commandes en base de données.

ProductRepository

Requêtes sur les sweat-shirts : mises en avant et filtre par prix.

- `findFeatured(int $limit): array` - Sweat-shirts mis en avant sur la page d'accueil.
- `findByPriceRange(?PriceRange $range): array` - Liste de la boutique, éventuellement filtrée par fourchette de prix.

ResetPasswordRequestRepository

Stockage des demandes de réinitialisation de mot de passe.

- `createResetPasswordRequest(object $user, DateTimeInterface $expiresAt, string $selector, string $hashedToken): ResetPasswordRequestInterface` - Crée l'entité de demande de réinitialisation pour un utilisateur.

UserRepository

Accès aux utilisateurs en base de données.

- `upgradePassword(PasswordAuthenticatedUserInterface $user, string $newHashedPassword): void` - Ré-hache automatiquement le mot de passe si l'algorithme de hachage évolue.

Security

AccessDeniedHandler

Utilisateur connecté sans les droits suffisants (ex. un client sur /admin) : il est renvoyé vers l'accueil avec un message au lieu d'une page d'erreur 403.

- `handle(Request $request, AccessDeniedException $accessDeniedException): ?Response` - Ajoute un message d'erreur et redirige vers la page d'accueil.

EmailVerifier

Envoi et vérification du lien de confirmation d'inscription.

- `sendEmailConfirmation(User $user): void` - Envoie l'e-mail contenant le lien d'activation signé.
- `handleEmailConfirmation(Request $request, User $user): void` - Vérifie la signature du lien reçu et active le compte.

UserChecker

Bloque la connexion des comptes dont l'adresse e-mail n'a pas encore été confirmée.

- `checkPreAuth(UserInterface $user): void` - Appelé avant la vérification du mot de passe : refuse les comptes non activés.

Service

CartService

Panier de l'utilisateur, conservé en session pendant toute l'expérience d'achat.

- `add(Product $product, Size $size, int $quantity): void` - Ajoute un sweat-shirt dans la taille choisie.
- `remove(int $productId, Size $size): void` - Retire une ligne du panier (toutes les quantités de ce produit dans cette taille).
- `getItems(): array` - Lignes du panier avec leurs produits chargés depuis la base.
- `getTotalInCents(?array $items): int` - Montant total du panier en centimes.
- `count(): int` - Nombre d'articles (somme des quantités).
- `clear(): void` - Vide entièrement le panier (après un paiement confirmé).

CheckoutService

Orchestration d'un achat : panier -> commande -> paiement Stripe -> confirmation.

- `createOrderFromCart(User $customer): Order` - Transforme le panier en commande « en attente de paiement ».
- `startPayment(Order $order, string $successUrl, string $cancelUrl): string` - Crée la session de paiement Stripe et retourne l'URL vers laquelle rediriger le client.
- `confirmPayment(string $stripeSessionId, User $customer): Order` - Valide la commande au retour de Stripe : contrôle du paiement, décrément du stock, articles retirés du panier.
- `cancelOrder(Order $order): void` - Annule une commande qui ne pourra pas être payée (erreur lors de la création du paiement).
- `cancel(string $stripeSessionId, User $customer): void` - Annule une commande en attente quand le client revient de Stripe sans payer.

ProductImageUploader

Enregistre les images envoyées depuis le back-office dans public/images/products.

- `upload(UploadedFile $file): string` - Déplace le fichier envoyé et retourne son nouveau nom.
- `remove(?string $fileName): void` - Supprime une image envoyée précédemment (les images d'origine 1.jpeg à 10.jpeg sont conservées).

StripeService

Service de paiement Stripe.

- `isConfigured(): bool` - Indique si une clé secrète Stripe est renseignée.
- `isTestMode(): bool` - Indique si la clé utilisée est une clé du mode test (bac à sable).
- `createCheckoutSession(Order $order, string $successUrl, string $cancelUrl): Session` - Crée la session Stripe Checkout correspondant à une commande.
- `retrieveCheckoutSession(string $sessionId): Session` - Relit une session Checkout auprès de Stripe.
- `isCheckoutSessionPaid(string $sessionId): bool` - Vérifie auprès de Stripe que la session a été réglée. L'information vient de Stripe et non de l'URL de retour, qui pourrait être falsifiée.

Twig

PriceExtension

Filtres d'affichage des prix au format français (29,90 €). Ils évitent de dépendre de l'extension PHP intl.

- `formatEuros(string|int|float|null $amount): string` - Prix exprimé en euros (ex. valeur DECIMAL "29.90").
- `formatCents(int $cents): string` - Montant exprimé en centimes (ex. 2990).